Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Kullanıcı Girdileri ve Hareket

PyGame ile Oyun Geliştirme – Hafta 4

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

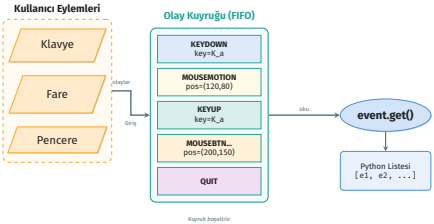
1. Event Sistemi
2. Klavye Girdileri
3. Fare Girdileri
4. Hareket Mekaniği
5. Özet ve Kapanış

Event Sistemi

Event Kuyruğu Nedir?

FIFO Kuyruk Yapısı

- First In, First Out – İlk giren, ilk çıkar
- Kullanıcı eylemleri (tuş, fare, pencere) kuyruğa eklenir
- `event.get()` kuyruğu okur ve boşaltır
- Varsayılan kapasite: 128 olay



Kuyruğu Okumayı Unutma!

`event.get()` çağırmazsan kuyruk dolar ve OS "yanıt vermiyor" der!

Event Nesnesi ve Özellikleri

pygame.event.Event Nesnesi

Her olay bir **Event** nesnesidir. Zorunlu özellik:

- `type` – Olayın türü (tam sayı)

Türe göre ek özellikler:

- `key` – Basılan tuş kodu
- `pos` – Fare konumu (x, y)
- `button` – Fare düğme numarası
- `mod` – Shift/Ctrl durumu
- `unicode` – Tuşun karakter karşılığı
- `rel` – Fare göreli hareketi

```
for olay in pygame.event.get():
    print(olay)
    print(olay.type)

    # Ozelliklerini gor
    print(olay.__dict__)
```

Keşfetme İpucu

`print(olay)` ile konsola yazdır. Hangi eylemin hangi olay türünü ve ek özellikleri ürettiğini gözlemle.

Olay Türleri Tablosu

Olay Türü	Tetikleyici	Ek Özellikler	Kullanım
`QUIT`	Pencere kapatma	–	Oyundan çıkış
`KEYDOWN`	Tuşa basma	`key, mod, unicode`	Tek seferlik giriş
`KEYUP`	Tuşu bırakma	`key, mod`	Tuş bırakma tespiti
`MOUSEBUTTONDOWN`	Fare tıklama	`pos, button`	Düğme etkileşimi
`MOUSEBUTTONUP`	Fare bırakma	`pos, button`	Sürükle-bırak
`MOUSEMOTION`	Fare hareketi	`pos, rel, buttons`	İmleç takibi
`MOUSEWHEEL`	Tekerlek kaydırma	`x, y`	Yakınlaştırma
`VIDEORESIZE`	Pencere boyutu	`size, w, h`	Yeniden boyutlama
`USEREVENT`	Özel olay	Kullanıcı tanımlı	Zamanlayıcı vb.

Her olay türü `pygame.` önekiyle kullanılır: `pygame.QUIT`, `pygame.KEYDOWN` vb.

Klavye Girdileri

KEYDOWN ve KEYUP Olayları

```
for olay in pygame.event.get():
    if olay.type == pygame.QUIT:
        calistir = False

    elif olay.type == pygame.KEYDOWN:
        if olay.key == pygame.K_SPACE:
            print("Bosluk basildi!")
        elif olay.key == pygame.K_ESCAPE:
            calistir = False

    elif olay.type == pygame.KEYUP:
        if olay.key == pygame.K_SPACE:
            print("Bosluk birakildi!")
```

KEYDOWN

- Tuşa **basıldığı an** tetiklenir
- Tek seferlik eylemler için ideal
- Örnek: ateş et, zıpla, menü seç

KEYUP

- Tuş **bırakıldığı an** tetiklenir
- Tuş basılı kaldığı süreyi ölçme
- Örnek: şarjı bırak, koşmayı durdur

Tuş Sabitleri

Sabit	Tuş
`K_UP`	Yukarı ok
`K_DOWN`	Aşağı ok
`K_LEFT`	Sol ok
`K_RIGHT`	Sağ ok
`K_SPACE`	Boşluk
`K_ESCAPE`	Escape
`K_RETURN`	Enter
`K_TAB`	Tab

Sabit	Tuş
`K_a` ... `K_z`	Harf tuşları
`K_0` ... `K_9`	Sayı tuşları
`K_LSHIFT`	Sol Shift
`K_RSHIFT`	Sağ Shift
`K_LCTRL`	Sol Ctrl
`K_F1` ... `K_F12`	Fonksiyon
`K_BACKSPACE`	Backspace
`K_DELETE`	Delete

Tüm sabitler `pygame.` önekiyle kullanılır: `pygame.K_SPACE`, `pygame.K_UP` vb.

key.get_pressed() ile Sürekli Girdi

```
# Her karede cagir (while dongusu icinde)
tuslar = pygame.key.get_pressed()

if tuslar[pygame.K_LEFT]:
    oyuncu_x -= 5
if tuslar[pygame.K_RIGHT]:
    oyuncu_x += 5
if tuslar[pygame.K_UP]:
    oyuncu_y -= 5
if tuslar[pygame.K_DOWN]:
    oyuncu_y += 5
```

Nasıl Çalışır?

- Tüm tuşların anlık durumunu döndürür
- Sonuç: bool değerli bir tuple
- `True` = tuş basılı
- `False` = tuş basılı değil
- Her karede sorgulanabilir

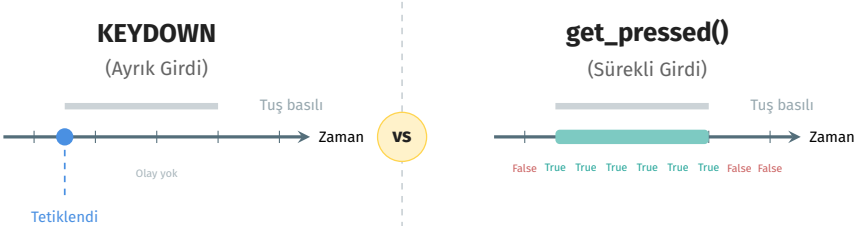
Dikkat: `if`, `elif` Değil!

`if` kullan ki çapraz hareket mümkün olsun. `elif` kullanırsan aynı anda sadece bir yön çalışır.

Ne Zaman Kullanılır?

Karakter hareketi gibi **tuş basılı kaldığı sürece** devam etmesi gereken eylemler.

Ayrık Girdi vs Sürekli Girdi



Tek seferlik eylemler:
Menü seçimi, ateş etme, zıplama başlatma

Sürekli eylemler:
Karakter hareketi, koşma, kamera dönme

Altın Kural

Tek seferlik eylem → KEYDOWN/KEYUP

Sürekli eylem → get_pressed()

Klavye ile Nesne Hareketi – Tam Örnek

```python
import pygame
pygame.init()
ekran = pygame.display.set_mode((800, 600))
saat = pygame.time.Clock()

oyuncu_x, oyuncu_y = 400, 300
HIZ = 5
KIRMIZI = (220, 50, 50)

calistir = True
while calistir:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            calistir = False

    tuslar = pygame.key.get_pressed()
    if tuslar[pygame.K_LEFT]:  oyuncu_x -= HIZ
    if tuslar[pygame.K_RIGHT]: oyuncu_x += HIZ
    if tuslar[pygame.K_UP]:    oyuncu_y -= HIZ
    if tuslar[pygame.K_DOWN]:  oyuncu_y += HIZ

    ekran.fill((30, 30, 50))
    pygame.draw.rect(ekran, KIRMIZI,
        (oyuncu_x, oyuncu_y, 40, 40))
    pygame.display.flip()
    saat.tick(60)

pygame.quit()
```

Adım Adım

1. Başlangıç konumu: merkez
2. Her karede tuşları sorgula
3. Basılı tuşa göre güncelle
4. Yeni konumda çiz

Dene!

Ok tuşlarıyla kırmızı kareyi hareket ettir. HIZ değerini değiştir.

Dikkat

Karakter ekrandan çıkabiliyor! Sınır kontrolü ile çözeceğiz.

Fare Girdileri

Fare Olayları

```
for olay in pygame.event.get():
    if olay.type == pygame.QUIT:
        calistir = False

    elif olay.type == \
            pygame.MOUSEBUTTONDOWN:
        # olay.pos = (x, y) konumu
        # olay.button = dügme no
        print(f"Tiklama: {olay.pos}")
        print(f"Dugme: {olay.button}")

        if olay.button == 1:  # Sol
            hedef_x, hedef_y = olay.pos
        elif olay.button == 3:  # Sag
            print("Sag tiklama!")
```

MOUSEBUTTONDOWN

- Fare düğmesine **basıldığında**
- `olay.pos` = tıklanan konum
- `olay.button` = düğme numarası
- Buton tıklama, hedef seçme, nesne yerleştirme

MOUSEBUTTONUP

- Fare düğmesi **bırakıldığında**
- Sürükle-bırak mekanikleri için
- Aynı özellikler: `pos`, `button`

Fare Düğmeleri Tablosu

button	Düğme
1	Sol düğme
2	Orta düğme (tekerlek)
3	Sağ düğme
4	Tekerlek yukarı
5	Tekerlek aşağı

Dikkat

Tekerlek kaydırma MOUSEBUTTONDOWN olarak da gelir (4/5). Modern PyGame'de ayrıca MOUSEWHEEL olayı var.

Yaygın Kullanımlar

Sol tıklama (1):

- Seçim, ateş etme, buton tıklama

Sağ tıklama (3):

- Özel yetenek, bağlam menüsü

Orta tıklama (2):

- Harita kaydırma, kamera döndürme

Tekerlek (4/5):

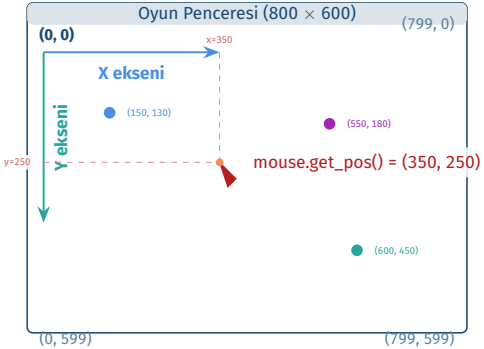
- Yakınlaştırma, silah değiştirme

MOUSEMOTION ve Fare Takibi

```
# Yontem 1: Olay tabanlı
for olay in pygame.event.get():
    if olay.type == pygame.MOUSEMOTION:
        fare_x, fare_y = olay.pos
        dx, dy = olay.rel   # goreli hareket

# Yontem 2: Dogrudan sorgulama
fare_x, fare_y = pygame.mouse.get_pos()

# Dügme durumu (her karede)
dugmeler = pygame.mouse.get_pressed()
if dugmeler[0]:   # Sol basiliysa
    print("Sol dügme basili!")
```



Performans Notu

MOUSEMOTION **çok sık** tetiklenir. Ağır işlem yapma, sadece konum kaydet.

Konum Sorgulama

- `olay.pos` = mutlak (x, y)
- `olay.rel` = fark (Δx, Δy)
- `mouse.get_pos()` = anlık konum

Fare ile Nesne Takibi – Nişangah Örneği

```
import pygame
pygame.init()
ekran = pygame.display.set_mode((800, 600))
saat = pygame.time.Clock()
pygame.mouse.set_visible(False)
BEYAZ, MAVI = (255,255,255), (50,120,220)

calistir = True
while calistir:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            calistir = False
    mx, my = pygame.mouse.get_pos()
    ekran.fill((20, 20, 30))
    pygame.draw.circle(ekran, MAVI,
        (mx, my), 20, 2)
    pygame.draw.line(ekran, BEYAZ,
        (mx-25, my), (mx+25, my), 1)
    pygame.draw.line(ekran, BEYAZ,
        (mx, my-25), (mx, my+25), 1)
    pygame.display.flip()
    saat.tick(60)
pygame.quit()
```

Nasıl Çalışır?

1. `set_visible(False)` ile imleci gizle
2. `get_pos()` ile konum al
3. Daire + artı ile nişangah çiz

Genişlet!

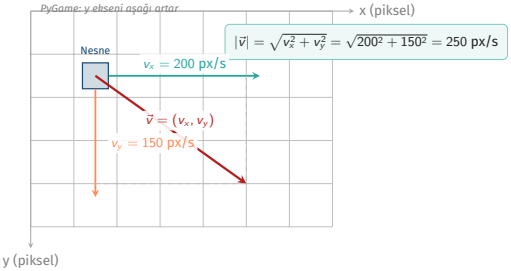
Sol tıkla → ateş efekti. Hedef nesneler ekle. Skor sayacı koy.

Hareket Mekaniği

Hız Vektörü

Hız = Yön + Büyüklük

- vx = yatay hız bileşeni
- vy = dikey hız bileşeni
- Pozitif vx → sağa hareket
- Pozitif vy → aşağı hareket



Konum Güncelleme

Her karede:

```
konum_x += vx
konum_y += vy
```

veya delta time ile:

```
konum_x += vx * dt
konum_y += vy * dt
```

Hatırlatma

PyGame'de y ekseni **aşağı** doğru artar! Yukarı gitmek = vy negatif.

Delta Time Hatırlatma

```
saat = pygame.time.Clock()
HIZ = 200  # piksel/saniye

while calistir:
    # dt: saniye cinsinden gecen sure
    dt = saat.tick(60) / 1000.0

    tuslar = pygame.key.get_pressed()
    if tuslar[pygame.K_RIGHT]:
        oyuncu_x += HIZ * dt
    if tuslar[pygame.K_LEFT]:
        oyuncu_x -= HIZ * dt
    if tuslar[pygame.K_DOWN]:
        oyuncu_y += HIZ * dt
    if tuslar[pygame.K_UP]:
        oyuncu_y -= HIZ * dt
```

Neden Delta Time?

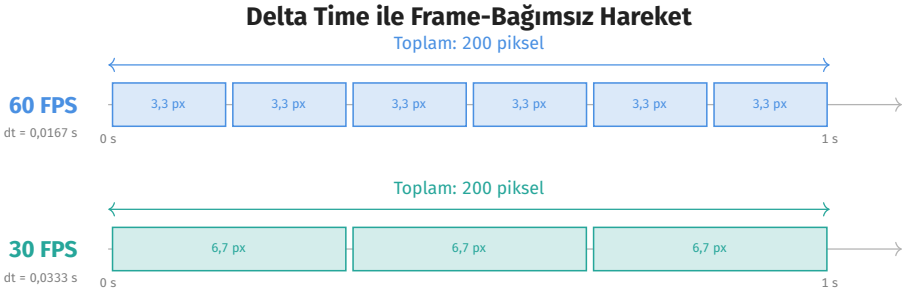
- `tick(60)` ms cinsinden süre döndürür
- 1000'e böl → saniye
- Hız: piksel/saniye olarak tanımla
- 200 px/s= her saniye 200 piksel

Frame-Bağımsız Hareket

dt kullanmadan: 30 FPS'de yavaş, 120 FPS'de hızlı

dt kullanarak: Her bilgisayarda **aynı hız**

Delta Time – Görsel Karşılaştırma



hız × dt = sabit mesafe/saniye → 200 × dt = 200 px/s

Farklı FPS'lerde aynı sürede aynı mesafe: **hız × dt = sabit mesafe/saniye**

Sınır Kontrolü Stratejileri

Clamp (Sıkıştır)

Kenara ulaşınca durur. Ekrandan çıkamaz.

```
x = max(0, x)
x = min(GEN-w, x)
```

Kullanım:
Platform, UI

Wrap (Sar)

Bir kenardan çıkınca karşıdan girer.

```
if x > GEN:
  x = 0
if x < 0:
  x = GEN
```

Kullanım:
Asteroids, Pac-Man

Bounce (Sekme)

Kenara çarptığında hız yönü tersine döner.

```
if x > GEN:
  vx = -vx
```

Kullanım:
Pong, Breakout

Strateji Seçimi

Oyunun türüne göre en uygun stratejiyi seç. Bir oyunda birden fazla strateji kullanabilirsin!

Sınır Kontrolü – Clamp Örneği

```
GEN, YUK = 800, 600
ekran = pygame.display.set_mode((GEN, YUK))
saat = pygame.time.Clock()
x, y = 400.0, 300.0
BOYUT, HIZ = 40, 250  # piksel/saniye

calistir = True
while calistir:
    dt = saat.tick(60) / 1000.0
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            calistir = False

    tuslar = pygame.key.get_pressed()
    if tuslar[pygame.K_LEFT]:  x -= HIZ * dt
    if tuslar[pygame.K_RIGHT]: x += HIZ * dt
    if tuslar[pygame.K_UP]:    y -= HIZ * dt
    if tuslar[pygame.K_DOWN]:  y += HIZ * dt

    # Sinir kontrolu (Clamp)
    x = max(0, min(x, GEN - BOYUT))
    y = max(0, min(y, YUK - BOYUT))

    ekran.fill((30, 30, 50))
    pygame.draw.rect(ekran, (50,200,80),
        (int(x), int(y), BOYUT, BOYUT))
    pygame.display.flip()
```

Clamp Mantığı

`max(0, min(x, GEN-B))`

- Sol/üst sınır: 0
- Sağ sınır: GEN – BOYUT
- Alt sınır: YUK – BOYUT

float Koordinat

Konum `float` tut, çizerken `int()` dönüştür. dt ile hassas hareket sağlanır.

Boyut Unutma!

Sağ/alt sınırda nesne boyutunu çıkar, yoksa kesilir.

Özet ve Kapanış

Bu Hafta Öğrendiklerimiz

Event Sistemi

- FIFO olay kuyruğu
- `event.get()`
- Event nesnesi
- `type`, `key`, `pos`

Klavye

- KEYDOWN / KEYUP
- Tuş sabitleri
- `get_pressed()`
- Ayrık vs Sürekli

Fare

- MOUSEBUTTONDOWN
- MOUSEMOTION
- `pos`, `button`
- `mouse.get_pos()`

Hareket

- Hız vektörü (vx, vy)
- Delta time
- Clamp / Wrap / Bounce
- Frame-bağımsızlık

Gelecek Hafta: Görseller ve Sprite Temelleri – Görsel dosyaları kullan, Sprite sınıfıyla tanış!

Mini Proje: Yılan Oyunu

Yılan Oyunu (Snake)

Bu haftanın tüm konularını birleştiren klasik oyun:

- Grid tabanlı hareket
- Ok tuşlarıyla yön kontrolü
- Yem toplama ve büyüme
- Çarpışma algılama + skor

Temel Mekanikler

1. Yılan her karede bir grid ilerler
2. KEYDOWN ile yön değiştirilir
3. Yem yenince kuyruk uzar
4. Kendine/duvara çarparsa biter

Öğrenme Hedefleri

- Klavye girdisi → yön değiştirme
- Liste → yılan gövdesi yönetimi
- Sınır kontrolü → duvar çarpışması

Dikkat

Yılan ters yöne gidemez! Sağa giderken sola basamaz. Bu kontrolü eklemeyi unutma.

Ödev

- Kitaptan Bölüm 4'ü okuyun
- Lab egzersizleri:
  1. Ok tuşlarıyla hareket eden karakter (clamp sınır kontrolü ile)
  2. Fare tıklamasıyla ekrana daire çizen program
  3. Nişangah uygulaması: hedef nesneler, tıklayınca skor artsın
  4. Yılan Oyunu'na ek özellik:
     - Hız zamanla artsın, ekranda puan gösterin
     - Game over ekranı (R ile yeniden başlat)

Kaynaklar

`pygame.org/docs/ref/event.html` – Event modülü
`pygame.org/docs/ref/key.html` – Tuş sabitleri
`pygame.org/docs/ref/mouse.html` – Fare modülü

Sorular?

Teşekkürler!